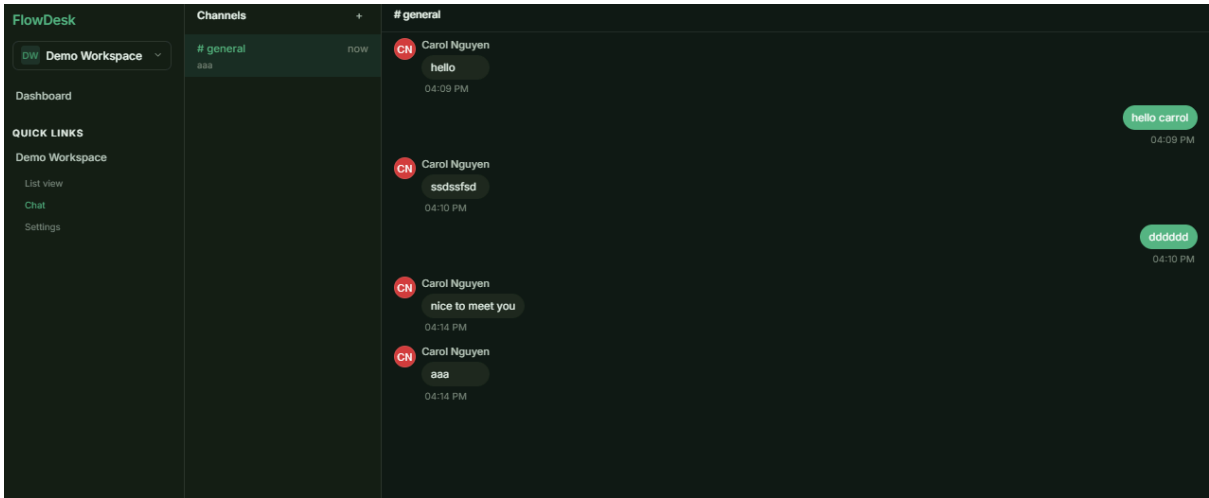
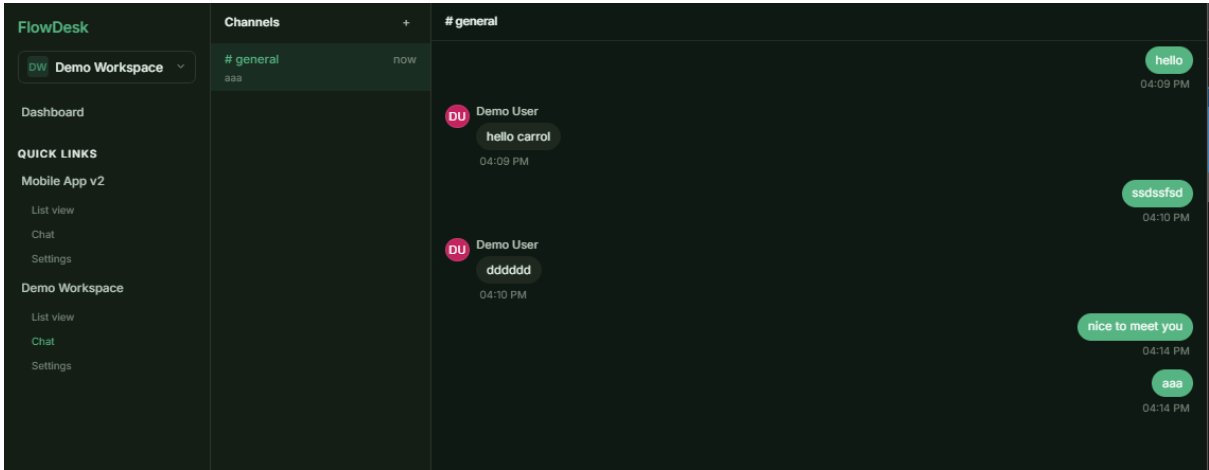


Sử dụng SocketIO để chat realtime

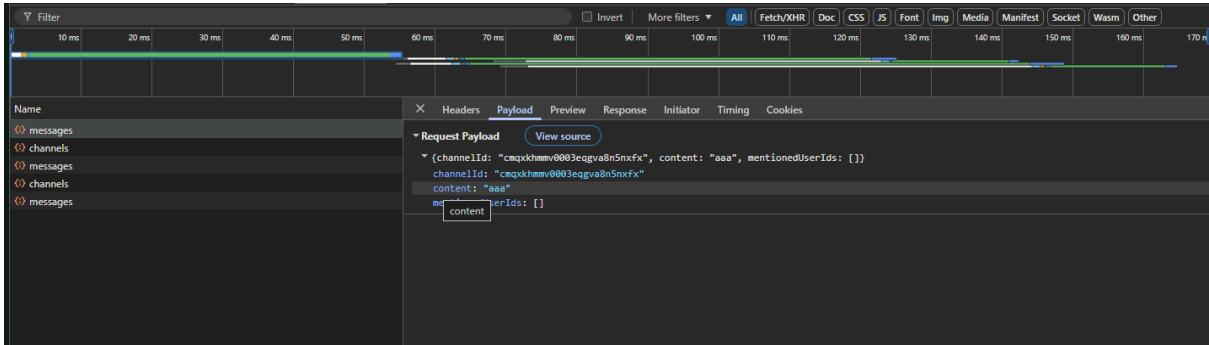
Demo User



Carrol



Payload socket



Sử dụng validation Cả FE và BE bằng Zod

apps/api/src/modules/task/task.routes.ts

```
taskRouter.post('/', async (c) => {
  const auth = c.get('auth');
  const body = createTaskSchema.parse(await c.req.json());
  return c.json({ task: await taskService.create(auth.user.id, body) },
    201);
});
```

packages/shared/src/task.ts

```
export const createTaskSchema = z.object({
  workspaceId: cuidSchema,
  columnId: cuidSchema,
  title: nameSchema,
  description: optionalString,
  priority: taskPrioritySchema.default('MEDIUM'),
  status: taskStatusSchema.default('TODO'),
  assigneeId: cuidSchema.nullable().optional(),
  dueDate: z.string().datetime({ offset: true }).nullable().optional(),
  parentTaskId: cuidSchema.nullable().optional(),
  position: z.number().int().min(0).optional(),
  labels: z.array(z.string().min(1).max(30)).max(20).optional(),
});
export type CreateTaskInput = z.infer<typeof createTaskSchema>;
```

apps/web/src/features/task/[api.ts](#)

```
import type { CreateTaskInput, Task, UpdateTaskInput } from
  '@flow-desk/shared/task';
import { api } from '@lib/api';

export const taskApi = {
  create(body: CreateTaskInput) {
    return api<{ task: Task }>('/api/tasks', {
      method: 'POST',
      json: body,
    });
  },
  move(taskId: string, body: { columnId: string; position: number;
    version: number }) {
    return api<{ task: Task }>(`/api/tasks/${encodeURIComponent(taskId)}/move`, {
```

```
        method: 'POST',
        json: body,
    });
},
update(taskId: string, body: UpdateTaskInput) {
    return api<{ task: Task
}>(`/api/tasks/${encodeURIComponent(taskId)}`, {
    method: 'PATCH',
    json: body,
    });
},
delete(taskId: string) {
    return api<{ ok: boolean
}>(`/api/tasks/${encodeURIComponent(taskId)}`, {
    method: 'DELETE',
    });
},
restore(taskId: string) {
    return api<{ task: Task
}>(`/api/tasks/${encodeURIComponent(taskId)}/restore`, {
    method: 'POST',
    });
},
};
```

Phân tích: Tái sử dụng Zod giữa Frontend và Backend trong flow-desk

Tổng quan kiến trúc

`packages/shared` là package trung tâm — **single source of truth** cho toàn bộ Zod schemas và TypeScript types. Cả `apps/api` (Hono + Node.js) lẫn `apps/web` (React + Vite) đều import từ đây, không ai tự define type riêng.

```
pnpm monorepo / flow-desk
├── packages/
│   ├── shared/      ← @flow-desk/shared — Zod schemas + TypeScript types
│   ├── db/          ← Prisma client
│   └── env/          ← Environment validation
├── apps/
│   ├── api/         ← Hono + Node.js — import schema + parse()
│   └── web/          ← React + Vite — import type only
```

`packages/shared` — Cấu trúc

Tổ chức file

```
packages/shared/src/
├── common.ts        ← Primitives tái dùng trong mọi domain
├── user.ts
├── auth.ts
├── task.ts
├── workspace.ts
├── comment.ts
├── chat.ts
├── notification.ts
├── notification-preferences.ts
├── attachment.ts
├── label.ts
├── pagination.ts
└── index.ts         ← Re-export tất cả
```

Subpath exports (`package.json`)

Package export từng subpath riêng biệt, giúp tree-shaking tốt hơn:

```
{
  "exports": {
    "./auth": { "types": "./dist/auth.d.ts", "import": "./dist/auth.mjs" },
    "./task": { "types": "./dist/task.d.ts", "import": "./dist/task.mjs" },
    "./workspace": { "types": "./dist/workspace.d.ts", "import": "./dist/workspace.mjs" },
    "./pagination": { "types": "./dist/pagination.d.ts", "import": "./dist/pagination.mjs" }
  }
}
```

Lớp 1 — Primitives (common.ts)

Building block nhỏ nhất, được tái dùng trong mọi domain:

```
// packages/shared/src/common.ts
import { z } from 'zod';

export const emailSchema = z.string().email().max(255).toLowerCase();

export const passwordSchema = z
  .string()
  .min(8, 'Password must be at least 8 characters')
  .max(128)
  .regex(/[A-Z]/, 'Password must contain at least one uppercase letter')
  .regex(/[a-z]/, 'Password must contain at least one lowercase letter')
  .regex(/[0-9]/, 'Password must contain at least one digit');

export const nameSchema = z.string().min(1).max(100).trim();
export const cuidSchema = z.string().min(20).max(30);
export const colorHexSchema = z
  .string()
  .regex(/^#([A-Fa-f0-9]{6}|[A-Fa-f0-9]{3})$/, 'Invalid hex color');

export const paginationSchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().min(1).max(100).default(20),
});
```

Lớp 2 — Domain schemas (task.ts)

Compose từ primitives, define toàn bộ CRUD schemas cho một domain:

```
// packages/shared/src/task.ts
import { z } from 'zod';
import { cuidSchema, nameSchema, optionalString, paginationSchema } from './common';

export const taskPrioritySchema = z.enum(['LOW', 'MEDIUM', 'HIGH', 'URGENT']);
export type TaskPriority = z.infer<typeof taskPrioritySchema>;
```

```

export const taskStatusSchema = z.enum([
  'BACKLOG', 'TODO', 'IN_PROGRESS', 'IN_REVIEW', 'DONE', 'BLOCKED',
]);
export type TaskStatus = z.infer<typeof taskStatusSchema>;

// Schema cho CREATE
export const createTaskSchema = z.object({
  workspaceId: cuidSchema,
  columnId: cuidSchema,
  title: nameSchema,
  description: optionalString,
  priority: taskPrioritySchema.default('MEDIUM'),
  status: taskStatusSchema.default('TODO'),
  assigneeId: cuidSchema.nullable().optional(),
  dueDate: z.string().datetime({ offset: true }).nullable().optional(),
  labels: z.array(z.string().min(1).max(30)).max(20).optional(),
});
export type CreateTaskInput = z.infer<typeof createTaskSchema>;

// Schema cho UPDATE (partial)
export const updateTaskSchema = z.object({
  title: nameSchema.optional(),
  priority: taskPrioritySchema.optional(),
  status: taskStatusSchema.optional(),
  version: z.number().int().min(0).optional(),
});
export type UpdateTaskInput = z.infer<typeof updateTaskSchema>;

// Schema cho response (bao gồm relations)
export const taskWithRelationsSchema = taskSchema.extend({
  assignee: z.object({
    id: cuidSchema,
    name: nameSchema,
    email: z.string().email(),
    avatarUrl: z.string().url().nullable(),
  }).nullable().optional(),
  _count: z.object({
    comments: z.number().int(),
    attachments: z.number().int(),
  }).optional(),
});
export type TaskWithRelations = z.infer<typeof taskWithRelationsSchema>;

```

Backend (Hono) — Cách dùng Zod

Pattern 1: `parse()` trực tiếp

Dùng trong `auth.routes.ts` — `parse` body và throw `ZodError` nếu invalid:

```
// apps/api/src/modules/auth/auth.routes.ts
import { registerSchema, loginSchema, refreshTokenSchema }
  from '@flow-desk/shared/auth';

authRouter.post('/register', async (c) => {
  const body = registerSchema.parse(await c.req.json());
  // body: RegisterInput — đã validated và typed
  const passwordHash = await bcrypt.hash(body.password, 10);
  const user = await prisma.user.create({
    data: { email: body.email, name: body.name, passwordHash },
  });
  // ...
});

authRouter.post('/login', async (c) => {
  const body = loginSchema.parse(await c.req.json());
  // body: LoginInput — email đã toLowerCase() theo emailSchema
  const user = await prisma.user.findUnique({ where: { email: body.email } });
  // ...
});
```

Pattern 2: zValidator() middleware

Dùng trong `task.routes.ts` — middleware validate trước khi vào handler:

```
// apps/api/src/modules/task/task.routes.ts
import {
  createTaskSchema, updateTaskSchema, moveTaskSchema,
  taskWithRelationsSchema,
} from '@flow-desk/shared/task';
import { zValidator } from '@hono/zod-validator';

// Validate query params với middleware
taskRouter.get('/',
  zValidator('query', listTasksQuerySchema, (result, c) => {
    if (!result.success)
      return c.json({ code: 'INVALID_QUERY', details: result.error.flatten() }, 400);
  }),
  async (c) => {
    const query = c.req.valid('query'); // fully typed, đã validated
    const { data } = await taskService.list(query, auth.user.id);
    // Serialize response — đảm bảo không leak field thừa
    return c.json({ data: z.array(taskWithRelationsSchema).parse(data) });
  }
);

// Validate request body
taskRouter.post('/', async (c) => {
  const auth = c.get('auth');
  const body = createTaskSchema.parse(await c.req.json());
  return c.json({ task: await taskService.create(auth.user.id, body) }, 201);
});
```

```
taskRouter.patch('/:id', async (c) => {
  const body = updateTaskSchema.parse(await c.req.json());
  return c.json({ task: await taskService.update(auth.user.id, id, body) });
});

taskRouter.post('/:id/move', async (c) => {
  const body = moveTaskSchema.parse(await c.req.json());
  return c.json({ task: await taskService.move(auth.user.id, id, body) });
});
```

Lưu ý: Schema còn dùng để **serialize response**: `taskWithRelationsSchema.parse(result)` đảm bảo BE không trả field thừa, client nhận đúng shape đã khai báo.

Frontend (React) — Cách dùng Zod

Frontend **không chạy Zod runtime** — chỉ import TypeScript types được infer từ schemas. Zero bundle overhead.

types.ts — Re-export type từ shared

```
// apps/web/src/features/task/types.ts
import type { Task } from '@flow-desk/shared/task';

export type { Task };
// Task = z.infer<typeof taskSchema> — không copy-paste, không lệch
```

api.ts — Typed request body từ shared

```
// apps/web/src/features/task/api.ts
import type { CreateTaskInput, Task, UpdateTaskInput }
  from '@flow-desk/shared/task';
import { api } from '@lib/api';

export const taskApi = {
  create(body: CreateTaskInput) {
    return api<{ task: Task }>('/api/tasks', { method: 'POST', json: body });
  },
  update(taskId: string, body: UpdateTaskInput) {
    return api<{ task: Task }>(`/api/tasks/${encodeURIComponent(taskId)}`, {
      method: 'PATCH',
      json: body,
    });
  },
};
```



```

delete(taskId: string) {
  return api<{ ok: boolean }>(`/api/tasks/${encodeURIComponent(taskId)}`, {
    method: 'DELETE',
  });
},
};

```

hooks.ts — useMutation typed hoàn toàn

```

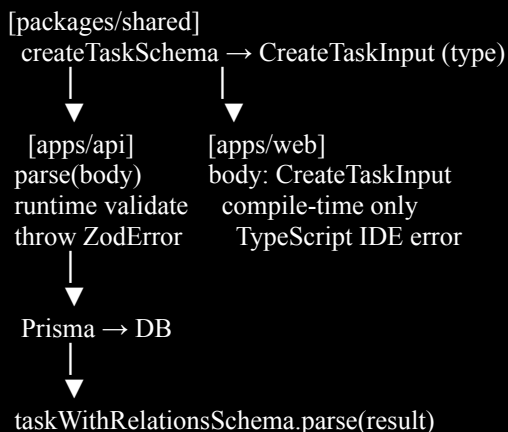
// apps/web/src/features/task/hooks.ts
import type { CreateTaskInput, UpdateTaskInput }
  from '@flow-desk/shared/task';
import { taskApi } from './api';

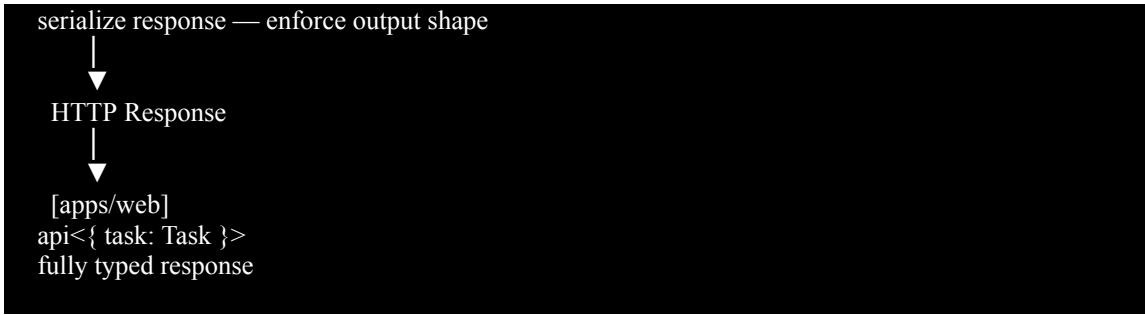
export function useCreateTask(workspaceId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (body: CreateTaskInput) => taskApi.create(body),
    onSuccess: () => qc.invalidateQueries({ queryKey: taskKeys.board(workspaceId) }),
  });
}

export function useUpdateTask(workspaceId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, body }: { id: string; body: UpdateTaskInput }) =>
      taskApi.update(id, body),
    onSuccess: () => qc.invalidateQueries({ queryKey: taskKeys.board(workspaceId) }),
  });
}

```

Luồng dữ liệu end-to-end





So sánh: Trước và sau pattern này

Tình huống	Không có shared schemas	Với @flow-desk/shared
Đổi field title → name	Sửa BE riêng, sửa FE riêng, dễ lệch nhau	Sửa 1 chỗ trong shared → TypeScript báo lỗi ngay ở mọi nơi
Thêm field mới vào API	Phải nhớ update cả 2 phía thủ công	Thêm vào schema → types tự propagate
API contract enforcement	Không có gì enforce	Schema là contract — compile-time + runtime
Validation rules	Có thể duplicate, FE validate khác BE	Rules tập trung 1 chỗ, không thể lệch
Bundle size FE	Zod chạy ở cả FE	Chỉ import type — zero runtime cost
Onboarding developer mới	Phải đọc cả FE lẫn BE code	Đọc packages/shared là hiểu toàn bộ API contract

Điểm đáng chú ý thêm

`changePasswordSchema` dùng `.refine()` để cross-field validation ngay trong shared — logic không bị duplicate giữa FE và BE:

```
// packages/shared/src/user.ts
export const changePasswordSchema = z
  .object({
```

```

currentPassword: z.string().min(1),
newPassword: passwordSchema,
})
.refine((data) => data.currentPassword !== data.newPassword, {
  message: 'New password must differ from current password',
  path: ['newPassword'],
});

```

`isoDateSchema` dùng `.transform()` để normalize Date → string, đảm bảo JSON serialization nhất quán giữa mọi môi trường:

```

const isoDateSchema = z
  .union([z.string(), z.date()])
  .transform((v) => (v instanceof Date ? v.toISOString() : v));

```

`listTasksQuerySchema` extend từ `paginationSchema` — compose schema thay vì copy:

```

export const listTasksQuerySchema = paginationSchema.extend({
  workspaceId: cuidSchema,
  status: taskStatusSchema.optional(),
  sortBy: z.enum(['createdAt', 'updatedAt', 'dueDate', 'priority', 'position']).default('position'),
  sortOrder: z.enum(['asc', 'desc']).default('asc'),
});

```